

Parallel Kernel Image Processing

Parallelizzazione e vettorizzazione di filtri di convoluzione su
immagini

Lorenzo Cappellini

Parallel Computing - A.A. 2025/2026

Indice

1	Introduzione	3
1.1	Obiettivo	3
1.2	Kernel di convoluzione	3
1.3	Approcci utilizzati	4
2	Implementazione	4
2.1	Flusso del programma	4
2.2	Classi	4
2.3	Configurazioni	5
2.3.1	Baseline sequenziale	5
2.3.2	Parallelizzazione con OpenMP	5
2.3.3	Vettorizzazione con AVX2	5
2.3.4	GPU computing con CUDA	5
2.3.5	AoS vs SoA	6
3	Analisi delle performance	6
3.1	Setup hardware	6
3.2	Metrica di valutazione	6
3.3	Dataset	7
3.4	AoS vs SoA	7
3.5	Confronto tempi di esecuzione	8
3.6	Confronto Speedup	8
4	Conclusioni	9

1 Introduzione

1.1 Obiettivo

Il progetto è stato sviluppato per l'assignment del corso di Parallel Computing, il quale richiede l'implementazione di un programma che sfrutti almeno uno degli approcci di parallelizzazione o vettorizzazione presentati durante il corso (OpenMP, vettorizzazione SIMD, CUDA), a partire da una versione sequenziale dello stesso algoritmo, corredata da un'analisi delle performance in termini di *speedup*.

Il problema scelto è il **Kernel Image Processing**, ovvero l'applicazione di un filtro di convoluzione 2D a un'immagine. Dato che ogni pixel in output è indipendente dall'altro, questa task si presta bene sia alla vettorizzazione SIMD che all'esecuzione parallela con OpenMP e su GPU.

Il codice sorgente è disponibile su GitHub al seguente indirizzo:
<https://github.com/lcappellini/ParallelKernelImageProcessing>.

1.2 Kernel di convoluzione

La convoluzione 2D consiste nell'applicare a ciascun pixel dell'immagine una combinazione pesata dei pixel vicini, secondo una matrice di coefficienti detta appunto "Kernel". Dato un kernel di dimensione $K \times K$, il pixel di output è calcolato con la seguente formula:

$$O(x, y) = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} I(x+i-r, y+j-r) K(i, j) \quad (1)$$

dove I rappresenta l'immagine di input, K il kernel di convoluzione e $r = \lfloor K/2 \rfloor$ il raggio del kernel. Per pixel appartenenti al bordo dell'immagine, le coordinate che fuoriescono dall'immagine vengono ricondotte al pixel di bordo più vicino (*clamping*). Assumendo una dimensione del kernel costante, il costo computazionale dell'algoritmo cresce linearmente con il numero N di pixel dell'immagine, ovvero $O(N)$.

La convoluzione permette di realizzare diversi filtri d'immagine, tra cui *sharpening*, *gaussian blur*, *edge detection kernel* ed *embossing*. In questo progetto è stato utilizzato un kernel di *edge detection*, applicato indipendentemente ai tre canali RGB.



Figura 1: Immagine di partenza

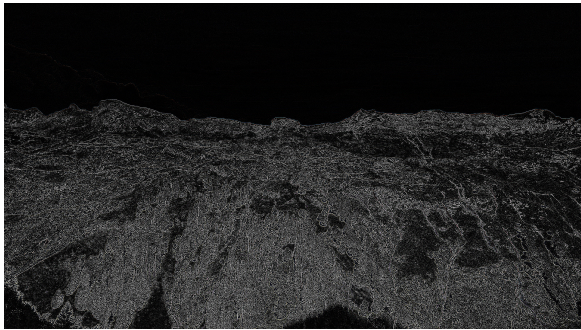


Figura 2: Immagine di output dopo l'applicazione del kernel di *edge detection*

1.3 Approcci utilizzati

Sono state implementate e confrontate le seguenti configurazioni, tutte derivate dallo stesso algoritmo di convoluzione:

- **Base**: versione sequenziale, quindi senza nessuna parallelizzazione né vettorizzazione esplicita.
- **OpenMP**: parallelizzazione su più thread del doppio ciclo sui pixel dell'immagine.
- **AVX2**: vettorizzazione delle istruzioni, che permette di eseguire quindi più operazioni con una singola istruzione.
- **OpenMP + AVX2**: combinazione delle due tecniche precedenti.
- **CUDA**: implementazione GPU con caricamento a tile in *shared memory*.
- **AoS vs SoA**: confronto tra layout di memoria *Array of Structures* e *Structure of Arrays*. Entrambi sfruttano la vettorizzazione AVX2, al fine di valutare l'influenza del layout dei dati in memoria sull'efficienza della convoluzione vettorizzata.

2 Implementazione

2.1 Flusso del programma

Per ciascuna configurazione, il programma:

1. carica tutte le immagini `.bmp` presenti nella cartella di lavoro con prefisso `in_`;
2. per ciascuna immagine e ciascuna modalità di calcolo prevista, applica un filtro (in questo caso un kernel di *edge detection*);
3. misura il tempo di esecuzione tramite `std::chrono::high_resolution_clock`; usando `std::chrono::milliseconds` si ottenevano errori di troncamento troppo elevati relativamente a tempi di esecuzione troppo brevi (1-2 ms con CUDA), perciò è stato necessario utilizzare `std::chrono::microseconds` (nei grafici i tempi sono riconvertiti in ms);
4. ripete l'esecuzione e la misurazione più volte (in questo caso 50), per ovviare ad eventuali fluttuazioni dovute ad altre task/processi del sistema operativo;
5. salva le misurazioni su file `.csv`.

2.2 Classi

Per modellare il problema sono state create le seguenti classi:

- **Image**: rappresenta un'immagine su cui operare e contiene dati come larghezza, altezza e numero di canali. Si occupa del caricamento e del salvataggio delle immagini (per semplicità è stato implementato soltanto il formato `.bmp`). L'immagine è salvata in layout **AoS** ed è possibile accedervi direttamente con un puntatore. Quando richiesto è possibile ottenere una copia convertita in layout **SoA**.
- **Kernel**: rappresenta il kernel di convoluzione, una matrice memorizzata come array lineare di float.
- **ImageEditor**: classe statica che implementa la logica di convoluzione vera e propria, in tutte le sue varianti. Espone metodi di applicazione diretta di effetti (come `edge_detection_effect`), che creano automaticamente i corrispettivo kernel e invocano il metodo `convolve` con la modalità richiesta (**AoS**, **SoA** o **CUDA**).

2.3 Configurazioni

Il programma è caratterizzato in più configurazioni diverse, perciò si ottengono di fatto più file eseguibili a sé stanti, ognuno compilato con precise flag di compilazione e ottimizzazione.

2.3.1 Baseline sequenziale

Questa configurazione è compilata senza direttive di parallelismo, e di fatto implementa in maniera sequenziale. Essa costituisce il riferimento base (T_{seq}) rispetto al quale vengono calcolati gli speedup delle altre versioni.

2.3.2 Parallelizzazione con OpenMP

In questa configurazione la parallelizzazione è ottenuta annotando il ciclo esterno sulle righe dell'immagine con `#pragma omp parallel for`: ogni thread elabora un sottoinsieme di righe in modo da distribuire il carico di lavoro equamente.

Vista l'indipendenza dei pixel e delle operazioni, non c'è necessità di sincronizzazione tra i thread, operazione che rappresenta solitamente un notevole collo di bottiglia per le prestazioni.

2.3.3 Vettorizzazione con AVX2

La vettorizzazione AVX2 opera su gruppi di pixel adiacenti, applicando lo stesso coefficiente del kernel simultaneamente a più elementi tramite registri a 256 bit. Questo permette di eseguire la stessa operazione su più dati con una sola istruzione, riducendo quindi notevolmente i tempi di esecuzione. Per questa configurazione la "conversione" delle normali operazioni in quelle SIMD è affidata al compilatore, specificando il flag `/arch:AVX2`. È importante sottolineare che questo tipo di vettorizzazione deve essere supportata dalla CPU utilizzata (quasi tutte le CPU moderne).

2.3.4 GPU computing con CUDA

In questa configurazione, il programma, al posto della funzione di convoluzione `convolveAoS`, esegue una funzione che dopo aver preparato il device (GPU) e allocato la memoria necessaria, avvia l'esecuzione di un kernel CUDA.

In particolare l'implementazione CUDA (`convolution2DKernel_2batch`) utilizza uno schema a *tile* con *shared memory*: ogni blocco di thread carica in memoria condivisa una porzione dell'immagine di input, allargata di un alone (*halo*) pari al raggio del kernel, per poi calcolare i pixel di output del tile senza ulteriori accessi alla memoria globale. La dimensione del tile di output è fissata a $TILE_WIDTH \times TILE_WIDTH$ ed il blocco di thread lanciato ha la stessa dimensione del tile.

Dato che la zona da caricare (compreso l'*halo*) è pari a $(TILE_WIDTH + Mask_width - 1)^2$, alcuni thread devono caricare due elementi in due fasi distinte *2-batch loading*:

1. **Primo caricamento:** ogni thread del blocco carica un primo elemento in *shared memory*, calcolando le coordinate di destinazione a partire dall'indice del thread nel blocco.
2. **Secondo caricamento:** gli stessi thread, con un offset pari alla dimensione del blocco, caricano un secondo elemento, necessario a coprire le celle di *shared memory* non ancora riempite dalla prima fase. Solo i thread la cui destinazione ricade effettivamente dentro l'area di *shared memory* eseguono questa seconda scrittura.

In entrambe le fasi, le coordinate di input fuori dai limiti dell'immagine vengono gestite tramite *zero-padding*: ai pixel esterni all'immagine viene assegnato valore zero.

Per ottenere un'ulteriore miglione i puntatori all'immagine e alla mask sono dichiarati come `const __restrict__`: questo informa il compilatore che tali dati non vengono modificati durante l'esecuzione del kernel, in questo modo possono essere messi in una cache costante, velocizzando così i tempi di lettura.

2.3.5 AoS vs SoA

Per effettuare un confronto di prestazioni con la classica struttura dati in AoS (RGBRGBRGB...) usata per salvare i pixel delle immagini, la convoluzione è stata implementata anche con un approccio in SoA, quindi con i 3 canali colore divisi in 3 array diversi (RRR...; GGG...; BBB...). Come specificato nella sezione 2.2, nel programma i pixel dell'immagine sono salvati con layout AoS, e convertiti in SoA prima della convoluzione. Dato che questa "traduzione" è separata dall'operazione di computazione principale e dipende esclusivamente dalla scelta implementativa di salvataggio dei dati che viene fatta, nella misurazione dei tempi di esecuzione non viene considerata. Entrambe le implementazioni sono compilate con il flag `/arch:AVX2`, in modo da confrontare l'effetto del layout di memoria sulla vettorizzazione SIMD.

3 Analisi delle performance

3.1 Setup hardware

Componente	Modello
CPU	AMD Ryzen 7 3700X 8-Core Processor
RAM	2 x Corsair 8GB DDR4
GPU	NVIDIA RTX 3060 Ti
Sistema operativo	Windows 10

Tabella 1: Configurazione hardware/software utilizzata per i benchmark

I grafici riportati nella Sezione 3 sono stati generati a partire da questi file CSV tramite uno script Python dedicato.

3.2 Metrica di valutazione

Per confrontare tra loro le tecniche utilizzate, tutti i risultati ottenuti vengono espressi in termini di **speedup** rispetto alla configurazione sequenziale di base, oltre che di tempo assoluto. Lo speedup è definito come:

$$S = \frac{T_{seq}}{T_{par}} \quad (2)$$

dove T_{seq} è il tempo impiegato dalla configurazione sequenziale Base e T_{par} è il tempo impiegato dalla configurazione in analisi, a parità di dimensione dell'immagine in ingresso.

Ogni configurazione è stata eseguita 50 volte su ogni immagine e dai tempi ottenuti è stato preso il valore mediano: questo permette di ottenere una misurazione più fedele che ignori le oscillazioni dovute allo scheduling delle altre task del sistema operativo.

3.3 Dataset

I test sono effettuati su immagini di dimensioni diverse, e rispecchiano le risoluzioni standard 16 : 9. Queste sono generate a partire da una stessa immagine di partenza tramite uno script Python. In questo modo è possibile valutare le prestazioni su un numero crescente di pixel (da 23 mila a 130 milioni).

Nome	Risoluzione	Numero di pixel
nHD	640 × 360	230 400
qHD	960 × 540	518 400
HD	1280 × 720	921 600
WXGA	1366 × 768	1 049 088
HD+	1600 × 900	1 440 000
Full HD	1920 × 1080	2 073 600
QHD	2560 × 1440	3 686 400
4K UHD	3840 × 2160	8 294 400
5K	5120 × 2880	14 745 600
8K UHD	7680 × 4320	33 177 600
16K UHD	15360 × 8640	132 710 400

Tabella 2: Risoluzioni delle immagini utilizzate per i benchmark

3.4 AoS vs SoA

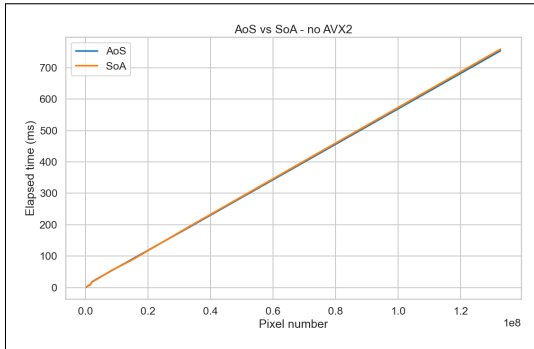


Figura 3: Tempi di esecuzione (ms) delle due implementazioni AoS e SoA, SENZA AVX2

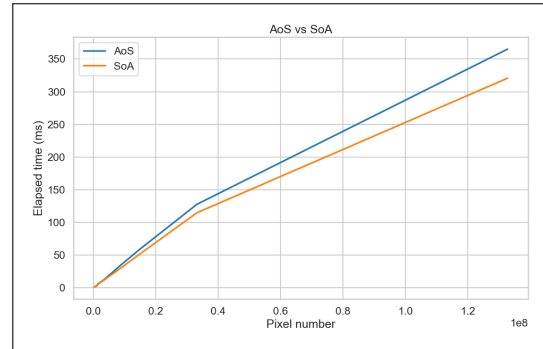


Figura 4: Tempi di esecuzione (ms) delle due implementazioni AoS e SoA

Il layout **SoA** è generalmente più favorevole laddove sono predominanti le operazioni sul singolo canale poiché i dati sono memorizzati in modo contiguo in memoria. Dunque, per questo problema specifico, la disposizione **SoA** in linea teorica non offre grandi vantaggi dato che l'operazione di convoluzione deve comunque essere eseguita su tutti i canali allo stesso modo.

Tuttavia con l'introduzione di AVX2, si nota una differenza: il layout **SoA** risulta più vantaggioso, poiché consente di caricare più efficientemente i valori dello stesso canale, dato che sono contigui in memoria. Nell'**AoS**, invece, i valori di uno stesso canale sono intervallati da quelli degli altri canali, rendendo necessarie operazioni di riordinamento aggiuntive.

3.5 Confronto tempi di esecuzione

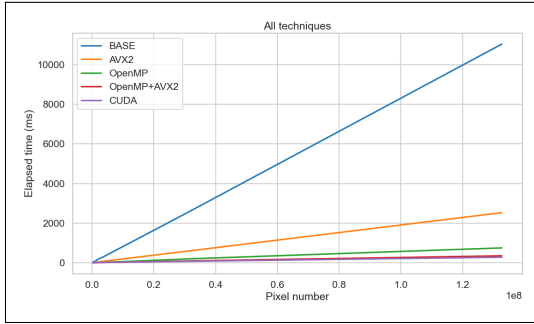


Figura 5: Grafico dei tempi impiegati da ogni configurazione

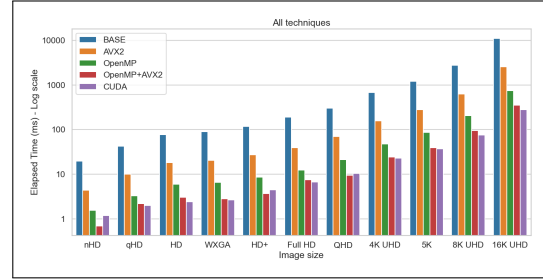


Figura 6: Grafico a barre dei tempi impiegati da ogni configurazione (scala logaritmica)

Il grafico (5) conferma che l'algoritmo di convoluzione ha costo $O(N)$, infatti il tempo di esecuzione cresce linearmente all'aumentare del numero di pixel per tutte le tecniche utilizzate. Come pronosticabile, l'implementazione Base è la più lenta. Inoltre la suddivisione del carico su più thread (OpenMP) è comunque superiore alla vettorizzazione di AVX2 e la combinazione delle due tecniche consente di sfruttare entrambi i livelli di parallelismo, ottenendo prestazioni superiori. Per avere un'ulteriore miglioramento infine c'è CUDA, anche se con immagini piccole la differenza è praticamente nulla.

3.6 Confronto Speedup

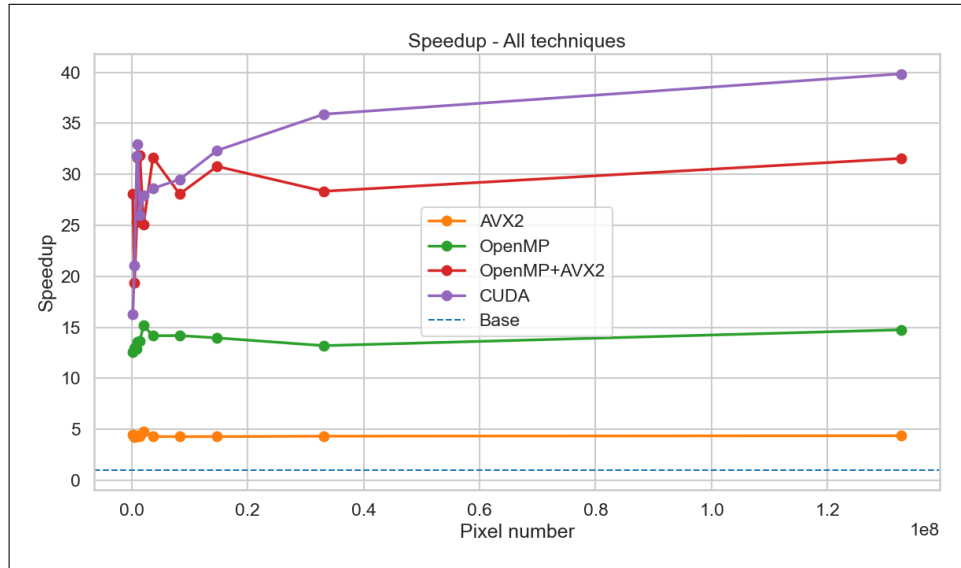


Figura 7: Grafico dei valori di speedup al variare del numero di pixel dell'immagine

Tra tutti questo grafico è senza dubbio il più significativo in quanto permette di mettere a confronto visivamente il rendimento di ogni tecnica di parallelismo adottata e di analizzare il miglioramento che si ottiene.

Come per i tempi assoluti, anche qui l'andamento è piuttosto lineare, ma è interessante notare che:

- per **AVX2** lo speedup è **costante**: eseguendo un certo numero (fisso) di operazioni in una sola, riduce il costo di ognuna di quel fattore, perciò lo speedup rimane tale anche al crescere dei pixel.
- per **OpenMP** lo speedup **cresce**: in questo caso sono presenti operazioni per creare il team di thread, scheduling e sincronizzazione della regione parallela; questo overhead ha un peso praticamente costante che si nota maggiormente quando lavora con un numero basso di pixel. Con un carico di lavoro superiore, la computazione stessa prevale, perciò l'**overhead diventa trascurabile**.
- per **CUDA** lo speedup ha un **andamento non lineare**: similmente a OpenMP, CUDA ha un **overhead**, ma molto più marcato a causa delle necessità di trasferimento dei dati tra host e device. Sono infatti indispensabili operazioni di allocazione (`cudaMalloc`), di copia (`cudaMemcpy`) e di "lancio" del kernel. Per quantità di dati minori (immagini piccole) il grande vantaggio della parallelizzazione sui thread della GPU viene quindi perso.

Per pixel count molto piccoli, sia CUDA che OpenMP+AVX2 mostrano grandi oscillazioni nello speedup, prima di stabilizzarsi su un andamento più regolare. Questo è dovuto al fatto che, a tempi di esecuzione dell'ordine del millisecondo, il rumore di misurazione ha un peso relativo molto più elevato rispetto al tempo di calcolo. Inoltre, dato che lo speedup è un rapporto tra due tempi, piccole variazioni assolute su T_{par} si traducono in variazioni percentuali molto ampie sul risultato finale.

4 Conclusioni

Nella maggior parte dei casi il campo di applicazione tipico del Kernel Image Processing è quello dell'editing di immagini reali, acquisite da fotocamere, e le risoluzioni sono cresciute negli ultimi anni. Anche per questo motivo e visti anche i risultati ottenuti, l'approccio con **CUDA** è senza dubbio la **scelta più efficace**.

Anche le altre tecniche però non sono irrilevanti: CUDA richiede una **GPU NVIDIA**, che potrebbe non esser disponibile sui dispositivi; in questi contesti **OpenMP+AVX2** rappresenta l'alternativa più solida, con uno speedup comunque notevole, operando soltanto su CPU e **senza dispositivi hardware aggiuntivi**. Anche **AVX2** da solo, pur con lo speedup più modesto, ha il vantaggio di richiedere **zero overhead** di setup ed essere applicabile senza coinvolgere altri thread, il che lo rende una scelta ragionevole quando la parallelizzazione **multithread non è praticabile**.